MSAS – Assignment #2: Modeling

Matteo Portantiolo, 250184

Exercise 1

To perform a preliminary analysis and evaluate a proposed satellite configuration, it is requested to model an active thermal control system and its associated mechanisms to assess the system's capability to maintain the temperatures within a certain range $[T_{min}; T_{max}]$. The satellite body is a rectangular cuboid (1.5 [m] height, 0.5 [m] side), with fixed solar panels (0.5 [m] side, 0.95 [m] length) located on the +Y-axis. The satellite flies with a fixed inertial attitude, with the satellite's Y-axis always directed towards the Sun. The satellite has two extendable radiators, which are located on the ±X-axis faces. The radiators are connected to the satellite body with electrically controlled hinges that can rotate along the Z-axis direction. The satellite with its radiators in open and closed configurations is represented in Figure 1.

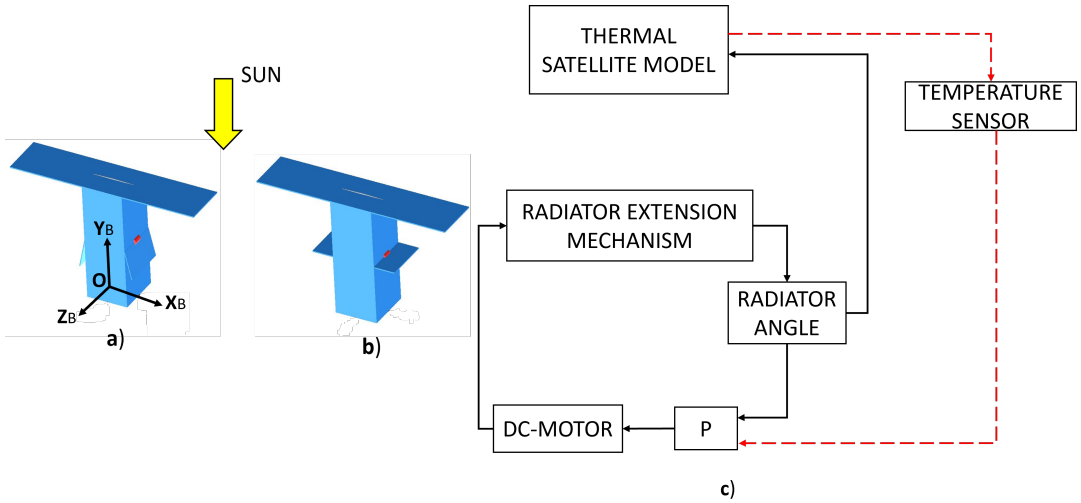

Figure 1: Satellite configuration: **a**) Closed radiator; **b**) Open radiator; **c**) Thermal control logic overview

The extension mechanism is based on a DC motor which gives the needed torque to move the radiator to the desired position. In Figure 2 the physical model of the mechanism is shown. The angle θ represents the rotation of the radiator with respect to the Z-axis and is measured counter-clockwise. For a rotation angle $\theta = 0$ deg, the radiators are in fully open configuration, for $\theta = -0.4\pi$ the radiators are in stowed (closed) configuration. The main parameters of the electro-mechanical model are reported in Table 1.

Parameter	Symbol	Value	Units
Resistance	R	0.1	Ω
Inductance	L	0.001	H
Motor constant	k_m	0.3	Nm/A
Radiator mass	m_r	0.2	kg
Radiator length	L_r	0.5	m

Table 1: Characteristics of the thermal control mechanism.

The radiator has a lower emissivity side, facing deep space in the closed configuration, and a higher emissivity side which is hidden when the radiator is in the stowed position. It is assumed

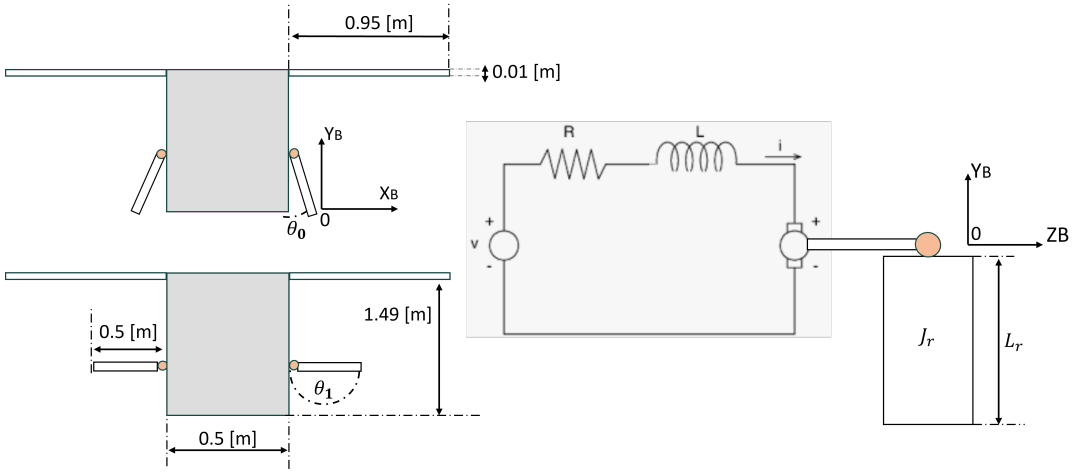

Figure 2: Schematics of the radiator extension mechanism

that radiative heat exchange among the satellite surfaces (re-radiation) is negligible and that no other thermal radiation sources (e.g., planets) are present. As a result, the thermal control of the satellite is regulated mainly by the radiator angle θ, as shown in Figure 1 c). By opening and closing the radiator, the effective area of the high- and low-emissivity sides exposed to deep space increases and decreases respectively. This results in a linear variation of the emissivity ϵ as a function of θ that can be expressed as in Eq. (1):

$$\epsilon(\theta) = \epsilon_{min} + \left[\frac{\epsilon_{max} - \epsilon_{min}}{0.4 \cdot \pi}\right](\theta(t) + 0.4 \cdot \pi) \tag{1}$$

When the radiator is in a closed configuration with $\theta(t) = -0.4\pi$ [rad] the emissivity is at its minimum $\epsilon_{min} = 0.01$. In fully open configuration instead, when $\theta(t) = 0$ [rad], the radiator emissivity reaches its maximum value $\epsilon_{max} = 0.98$. Using a lumped nodes thermal modeling, the satellite can be divided into 5 components (the two solar panels, the main body, and the two radiators). The radiators and solar panels are connected to the main body node by a conductive path, while all nodes are thermally radiating towards deep space (see Figure 3).

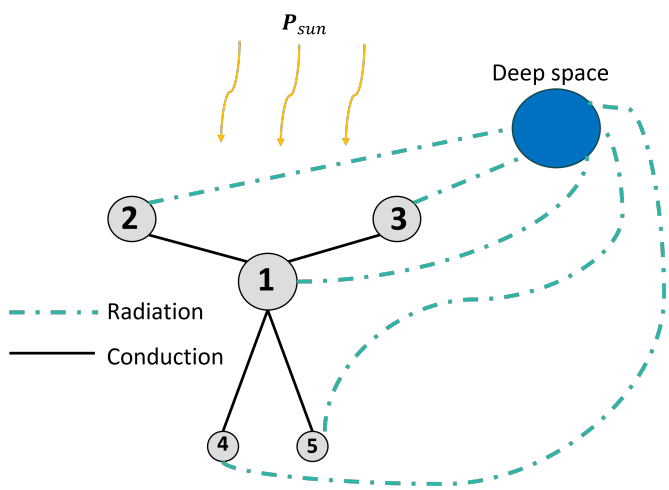

Figure 3: The thermal lumped nodes model

The solar radiation from the Sun is acting only on the two solar panel nodes and on the top face of the main-body node (the radiators are always in shadow). The temperature of deep space is constant and equal to T_{ds}= 3 [K]. The characteristics of the thermal network and its components are given in Table 2.

Parameter	Symbol	Value	Unit
Sun Power	P_{Sun}	1350	W/m^2
Heat capacity	C_1	1.5×10^5	J/K
Heat capacity	C_2, C_3	1187.5	J/K
Heat capacity	C_4, C_5	30	J/K
Thermal conductance	G_{12}, G_{13}, G_{14}, G_{15}	10	W/K
Absorptivity	α_1	0.6	-
Absorptivity	α_2, α_3	0.78	-
Emissivity	ϵ_1	0.45	-
Emissivity	ϵ_2, ϵ_3	0.75	-

Table 2: Thermal parameters of the system.

Part 1: causal modeling (9 points)

Considering the following main constitutive equations of the system:

$$C_i \frac{dT_i}{dt} = \sum_i^n (Q_{in}^i - Q_{out}^i) \tag{2}$$

$$V_{in} = k_p \cdot (T_1 - T_1^{ref}) \tag{3}$$

$$J_r \ddot{\theta} = \tau_{in} \tag{4}$$

where Eq. (2) is the conservation of energy at a thermal node; Eq. (3) is a proportional control law in which V_{in} is the input voltage applied on the DC-motor, and T_1^{ref} is the reference temperature of the main-body; and Eq. (4) is the simplified rotational radiator dynamics where the input torque is linked to current flowing into the motor through the relation: $\tau_{in} = k_m \cdot i$. Assuming that: the upper limit $\hat{T}_{max}$ and lower limit temperature $\hat{T}_{min}$ allowed for **node 1** are 300 [K] and 290 [K] respectively, the motor control is capable of maintaining the angle θ within the physical limits, all the thermal nodes start with a temperature of 298.15 [K], and that the radiator is closed at the beginning of the simulation ($\theta(0) = -0.4\pi$):

1. Formulate the full system of non-linear ODEs making explicit the state variables of the whole simulation.

2. Considering a simulation time of at least 50 [h], define the value for the proportional gain k_p such that: the temperature is kept around the target $T_1^{ref} = 294.15$ [K], and within $t = 10$ [h] the maximum temperature oscillations are less than 0.1% of T_1^{ref}. Plot the resulting temperature evolution over time on all thermal nodes.

3. Discuss the results and the ode used in Matlab for the integration.

Part 2: acausal modeling (6 points)

Using the Modelica standard library, reproduce in Dymola the physical model of the thermal-electro-mechanical system described above. You can build your **own model block** in Modelica to implement specific signals such as $\epsilon(\theta)$ or the control strategy. Please note that when you create a new model block (see in Table 3) you can insert input/output variables and your equations (see as an example the **Text View** of the block: *Modelica/Blocks/Math/Add*). Then, simulate it in OpenModelica comparing the results with the ones obtained using the causal model.

(15 points)

Physical element	*Modelica library*
Rigid-body	Modelica/Mechanics/MultiBody/Parts
Revolute joint	Modelica/Mechanics/MultiBody/Parts
Fixed-translational element	Modelica/Mechanics/MultiBody/Parts
DC-motor	Modelica/Electrical/Machines/BasicMachines/DCMachines
Thermal node	Modelica/Thermal/HeatTransfer/Components
Thermal resistances	Modelica/Thermal/HeatTransfer/Components
Logical switch	Modelica/Blocks/Logical/Switch
Hysteresis	Modelica/Blocks/Logical/Switch
Temperature sensor	Modelica/Thermal/HeatTransfer/Sensors
Your model block	File/New/Block

Table 3: Modelica libraries.

Point 1.1

The satellite thermal control system is modeled with a Lumped Parameter Approach (LPA), composed of five thermal nodes as shown in Figure 3: the main body, two solar panels, and two radiators. Firstly, it is important to specify the assumptions applied to the model and the equivalent surfaces used for the different nodes.

1. **Main Body**: node 1 receives solar radiation only on the top face ($A_1 = 0.25m^2$) while it radiates toward deep space (DS) with his total surface ($A_{tot} = 3.5m^2$).
2. **Solar Panels**: nodes 2 and 3 absorb solar radiation on the surface facing the Sun, while they emit toward DS with the opposite face ($A_2 = A_3 = 0.475m^2$). Furthermore, the width of the panels has been neglected.
3. **Radiators**: nodes 4 and 5 do not absorb solar radiation and radiate only from one face toward DS ($A_4 = A_5 = 0.25m^2$).

Now, it is useful to make explicit the state variable of the system:

$$\mathbf{x} = [T_1; T_2; T_3; T_4; T_5; \theta; \dot{\theta}; I]$$

where T_1 to T_5 are the temperatures of the five nodes, θ is the rotation angle of the radiator, $\dot{\theta}$ is the angular velocity of the radiator and I is the current in the DC motor.
Given the assumptions and the state vector, to formulate the full system of non-linear ODEs, the thermal, mechanical, and electrical domains of the problem are analyzed.

- **Thermal domain** Knowing the thermal properties of the network from Table 2, it is possible to write Eq. (2) for each thermal node, obtaining the first five ODEs of the system in Eq. (7). Each heat exchange term has a different contribution: the thermal power is positive when entering the system (solar radiation absorption), negative when leaving the systems (deep space emission). The conductive exchange, instead, is considered positive for the first node (main body), while negative for all other nodes (panels and radiators). It is important to note that the emissivity is constant for the main body and solar panels, but varies linearly with the rotation angle θ for the radiators, as dictated in Eq. (1).
- **Mechanical domain** The dynamics of the radiator must be properly modeled, knowing that the moment of inertia of the radiator is $J_r = \frac{1}{3} m_r L_r^2$ and the input torque is $\tau_{in} = k_m I$. Considering that the rotation angle of the radiator can be modeled as described in Eq. (4), the dynamics can be rewritten as a system of first-order ODEs in state variable, as shown in Eq. (5).

$$\begin{cases} \dot{\theta} &= \dot{\theta} \\ \ddot{\theta} &= \frac{\tau_{in}}{J_r} = \frac{k_m}{J_r} I \end{cases} \tag{5}$$

- **Electrical domain** Starting from Figure 2 and the characteristics listed in Table 1, it is possible to apply the Kirchhoff's voltage law as in Eq. (6).

$$V_{in} = V_R + V_L + V_m = RI + L\frac{dI}{dt} + k_m\dot{\theta} \tag{6}$$

  V_{in} is the input voltage applied to the DC motor and is governed by the proportional control law $V_{in} = k_p(T_1 - T_1^{ref})$ dictated in Eq. (3). Here, k_p is the proportional gain that needs to be tuned, and T_{ref} is the reference temperature.

The full system of non-linear ODEs governing the problem is then retrieved in Eq. (7).

$$\begin{cases} \frac{dT_1}{dt} &= \frac{1}{C_1}\left[\alpha_1 P_{Sun} A_1 - \varepsilon_1 \sigma A_{tot}\left(T_1^4 - T_{DS}^4\right) + G\sum_{i=2}^{n}\left(T_i - T_1\right)\right] \\ \frac{dT_2}{dt} &= \frac{1}{C_2}\left[\alpha_2 P_{Sun} A_2 - 2\varepsilon_2 \sigma A_2\left(T_2^4 - T_{DS}^4\right) - G\left(T_2 - T_1\right)\right] \\ \frac{dT_3}{dt} &= \frac{1}{C_3}\left[\alpha_3 P_{Sun} A_3 - 2\varepsilon_3 \sigma A_3\left(T_3^4 - T_{DS}^4\right) - G\left(T_3 - T_1\right)\right] \\ \frac{dT_4}{dt} &= \frac{1}{C_4}\left[-2\varepsilon_4(\theta) \sigma A_4\left(T_4^4 - T_{DS}^4\right) - G\left(T_4 - T_1\right)\right] \\ \frac{dT_5}{dt} &= \frac{1}{C_5}\left[-2\varepsilon_5(\theta) \sigma A_5\left(T_5^4 - T_{DS}^4\right) - G\left(T_5 - T_1\right)\right] \\ \frac{d\theta}{dt} &= \dot{\theta} \\ \frac{d\dot{\theta}}{dt} &= \frac{k_m}{J_r} I \\ \frac{dI}{dt} &= \frac{1}{L}\left[k_p\left(T_1 - T_1^{ref}\right) - RI - k_m\dot{\theta}\right] \end{cases} \tag{7}$$

Point 1.2

In order to integrate the system of ODEs, the right-hand side has been defined inside a Matlab function. For propagation, the initial temperature for all nodes T_i is assigned and equal to 298.15 K, the radiator angle θ is set to its closed position, $\dot{\theta}$ to zero, and the current I flowing in the circuit to zero.
To control the motion of the radiator, an active control mechanism has been implemented for the angle θ. This ensures that the radiator does not exceed the minimum or maximum allowed angles of opening and closing. If θ reaches the lower boundary, while the temperature of the first node is below the target value, then the control is switched off imposing a proportional gain equal to zero. This simulates null input voltage to the DC motor. The current does not go instantaneously to zero, but the transient is expected to be very fast. On the other hand, if θ reaches the upper boundaries, while T_1 is above the desired temperature, the control is switched off in the same way.
As for the proportional gain k_p, different considerations were made about its trial and error tuning process. First, a minimum value of $k_p^{min} = 4.92 \cdot 10^{-5}$ was carefully chosen to ensure that T_1 oscillations remained within 0.1% of the reference temperature T_{ref} after exactly 10 hours, minimizing excessive temperature fluctuations. Then, a maximum value of $k_p^{min} = 1 \cdot 10^{-4}$ was tuned to achieve faster convergence, but still avoiding extremely fast dynamics of the radiators, which could be damaged in a real mission. This value was found by arbitrary imposing 10-12 maximum oscillations of the radiators, as shown in Figure 4, where the evolution of θ and $\dot{\theta}$ are plotted.
It can be observed that, initially, the control logic opens the radiators to dissipate heat, because the initial temperature of the main body is higher than the target value. The maximum aperture

of zero degree is maintained for some time, during which the rate of change of θ is zero. At steady state, the radiator angle converges to $\theta = -36.06deg$, while $\dot{\theta}$ to zero, as expected.

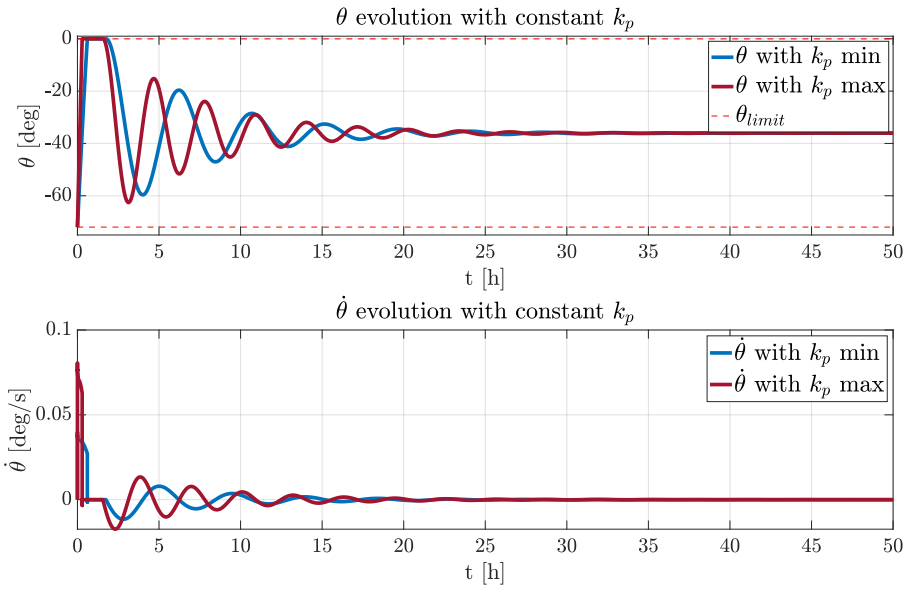

Figure 4: θ and $\dot{\theta}$ evolution with constant k_p

Figure 5, instead, reports the transient behavior of the main body temperature. As expected, the temperature corresponding to k_p^{min} enters the boundaries exactly at $t = 10h$ (green dot line). On the other hand, k_p^{max} guarantees a faster convergence within the temperature limits. Finally, the amplitude of oscillations becomes smaller and smaller as the controller converges to the target temperature.
Lastly, the temperature evolution of solar panels and radiators (nodes 2-3-4-5) is reported in Figure 6. Here, only small differences can be observed between the selection of the two k_p.

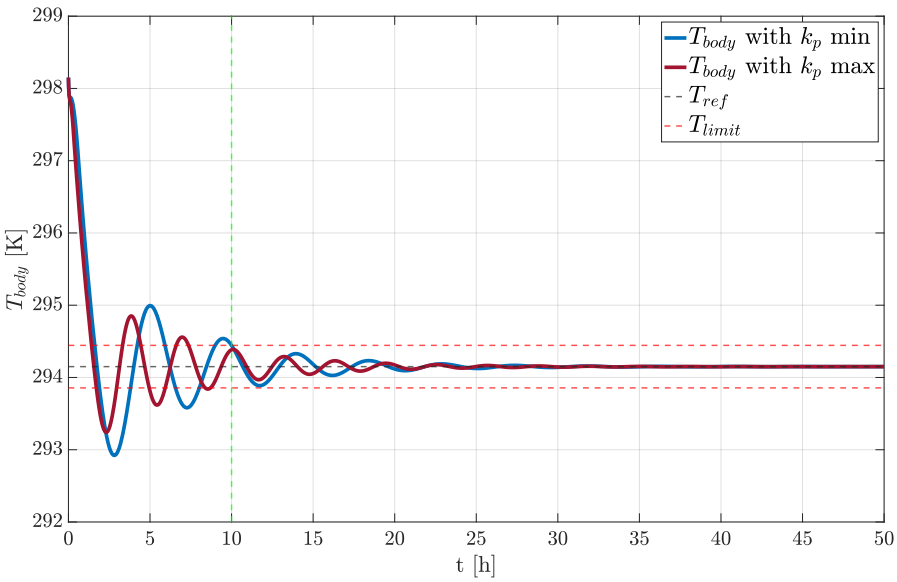

Figure 5: T_1 evolution with constant k_p

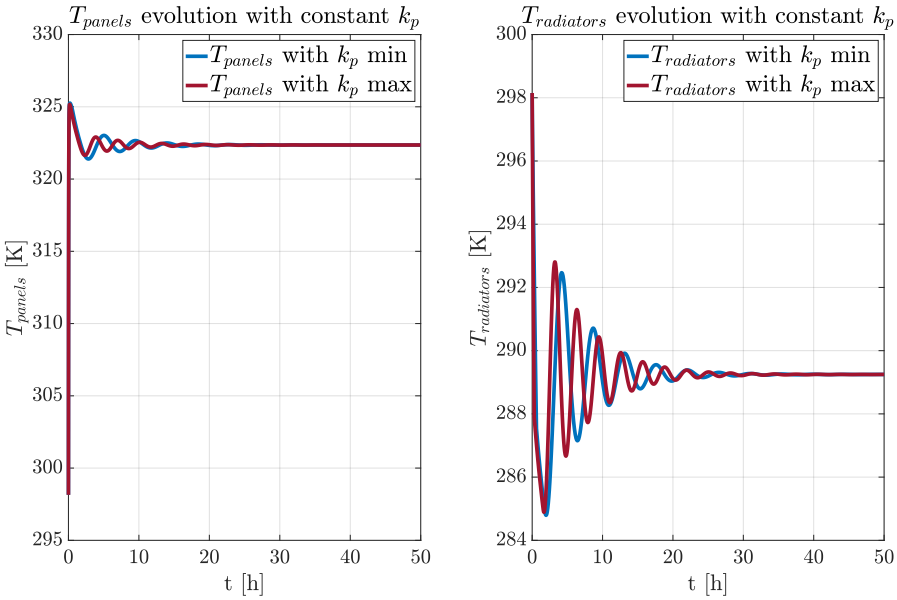

Figure 6: T_{2-5} evolution with constant k_p

Point 1.2.bis

A further analysis has been carried out to implement a control mechanism with adaptive proportional gain k_p. The algorithm allows to double k_p every time the main body temperature crosses the target T_1^{ref}, from a value of $k_p^0 = 1 \cdot 10^{-5}$ up to a maximum value of $k_p^{max} = 1 \cdot 10^{-4}$. This can be achieved through an event function that stops the integration, in a while loop, until the final instant or k_p^{max} is reached. This control logic ensures a smoother convergence of the main body temperature and more gradual radiator movements, as illustrated in Figure 7 and Figure 8.

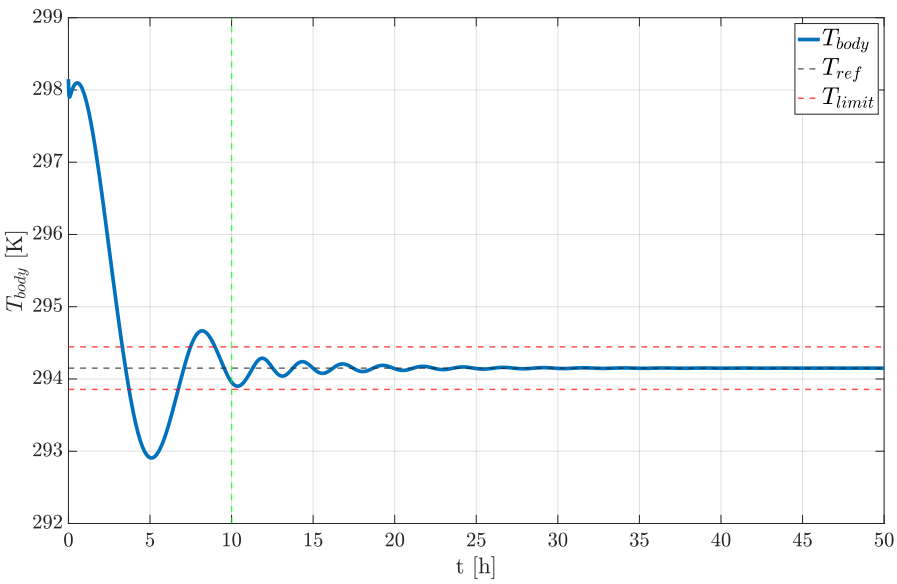

Figure 7: T_1 evolution with adaptive k_p

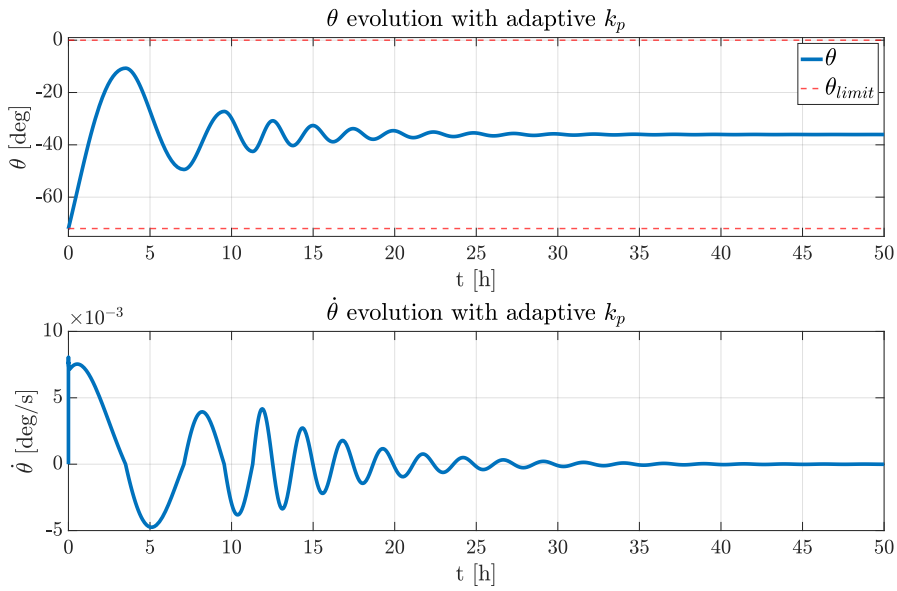

Figure 8: θ and $\dot{\theta}$ evolution with adaptive k_p

Point 1.3

The system of non-linear ODEs was solved using the Matlab© solver `ode15s`. This choice was driven by the stiffness of the system. In fact, stiffness is expected since multiple physical domains, characterized by different dynamics speed, are combined together. The circuit dynamics is expected to be much faster than the mechanical and thermal ones. Thus, the eigenvalues of the system will have real parts which differ by order of magnitude, leading to a stiff system by definition.

Point 2

The acausal modeling of the thermal-electro-mechanical system has been developed in Dymola exploiting built-in blocks from the Modelica libraries together with custom blocks. The model has been divided into three subsystems: thermal, electromechanical and control block. Figure 9 shows how they are correlated through various inputs and outputs, which will be discussed subsequently.

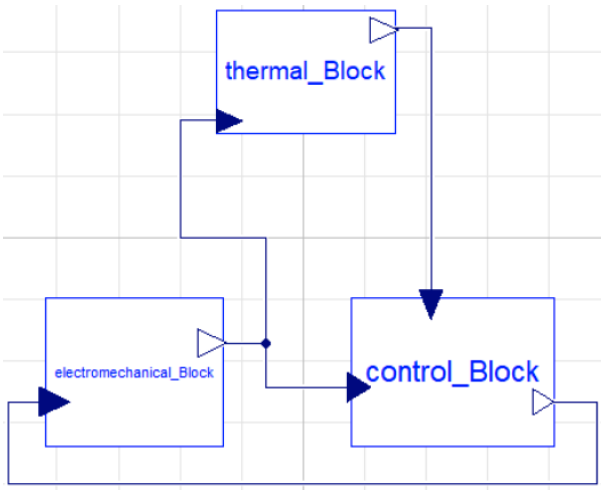

Figure 9: Thermal-electro-mechanical system blocks

Starting with the thermal domain, five lumped thermal nodes representing the main body, two solar panels, and two radiators were modeled using the *HeatCapacitor* block. The initial temperature conditions for each node were selected according to the specifications provided in the exercise.
To model the solar heat flux on the top face of the main body and solar panels, a *FixedHeatFlow* block was connected to the respective thermal nodes. For the conductive heat transfer between the solar panels, the radiators, and the main body, *ThermalConductor* blocks were employed. The radiative heat transfer was modeled using *BodyRadiation* blocks, which linked the main body and solar panels to deep space. Deep space was simulated using a *FixedTemperature* block, imposing a temperature of $3K$.
To accurately model the radiative heat transfer of the radiators, two custom blocks were developed and integrated into the system:

1. *Variable_G_r*: responsible for dynamically computing the radiative conductance G_r. This block receives the angle θ as input, which is provided by the electromechanical model. Based on θ, the block calculates the variable emissivity ε. Then it computes G_r with the following formula: $G_r = A \cdot \varepsilon$.

2. *Radiator_variable*: modified version of the standard *BodyRadiation* block to have as new input the variable radiative conductance G_r.

Finally, a *TemperatureSensor* block was used to monitor the temperature of the main body T_1. Figure 10 shows the whole model, highlighting the input θ from the electromechanical block and the output T_1.

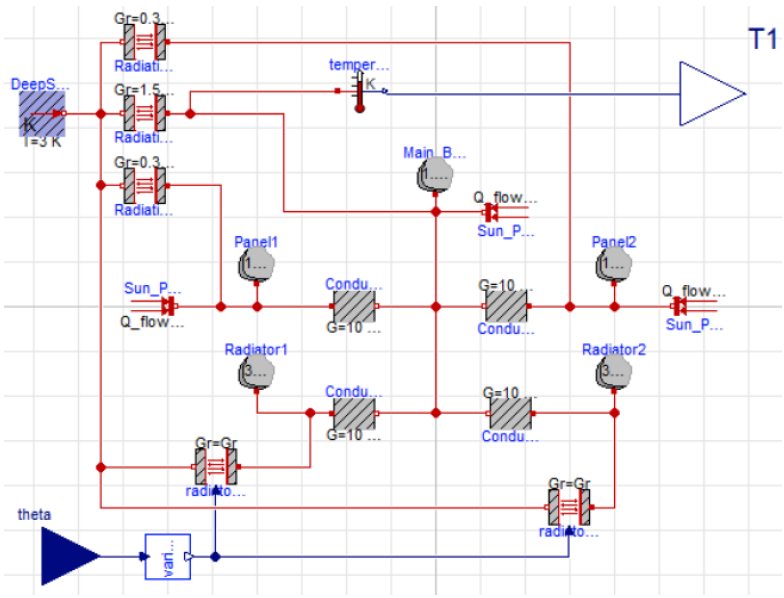

Figure 10: Thermal block

As the electromechanical domain, the circuit was modeled using components representing the electrical dynamics driving the connected mechanical load. Specifically, the *SignalVoltageSource* block was used to supply the input voltage from the control block, while the *Resistor* and *Inductor* blocks were included to model resistance and inductance in the circuit. The electromotive force was represented by the *ElectromotiveForce (EMF)* block. To simulate the rotational dynamics, the *Inertia* block was connected to the *EMF* output, representing the moment of inertia the mechanical load. An *AngleSensor* was placed at the end of the circuit to measure the angular displacement θ of the rotating system, representing the output of the electromechanical block, as presented in Figure 11.

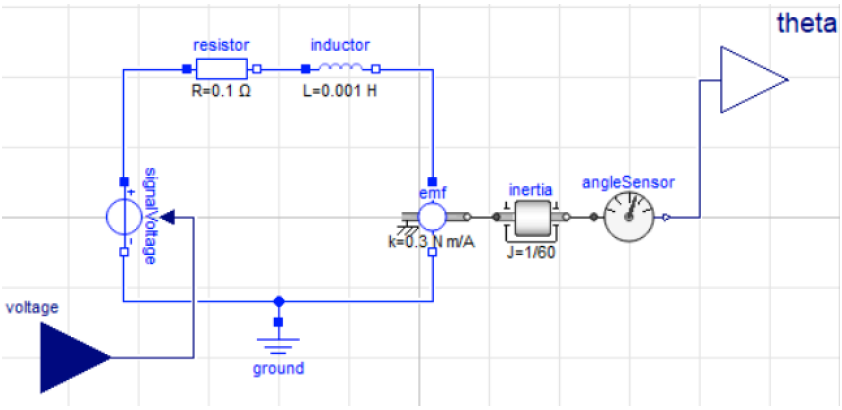

Figure 11: Electromechanical block

Finally, a switch model has been created to simulate the active control algorithm implemented in Matlab. The model uses logical conditions and signal processing blocks to represent the decision-making process. The input temperature T_1 and the angular position θ are monitored using comparison blocks (*lessEqual* and *greaterEqual*) to evaluate the required conditions. Logical operations are performed with an *AND* gate to determine when the control algorithm should be activated. The output of the logical conditions is connected to a *Switch* block, which modifies the system output based on the active state of the algorithm. The output of the controller V is proportional to the temperature difference between T_1 and T_{ref}, scaled by the proportional gain k_p set to $1 \cdot 10^{-4}$. Figure 12 presents the control logic network.

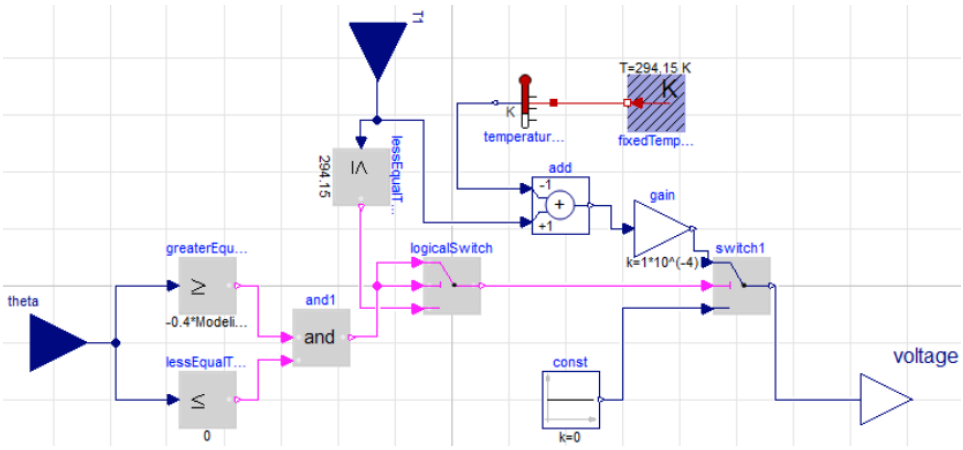

Figure 12: Control block

Once the model was completed in Dymola, the simulation was run in OpenModelica. The selected integration scheme was the default `DASSL` with a tolerance of $1 \cdot 10^{-12}$. This scheme has been chosen for its ability to manage stiff systems.
To compare the results obtained through causal and acausal modeling approaches, the simulation data were exported from OpenModelica in `.mat` format and plotted in Matlab. To validate the models, the approach with the maximum value of the constant proportional gain k_p^{max} was selected from Matlab, since this value is the same used in the Dymola simulation. The comparison for each state variable is presented in Figure 13.

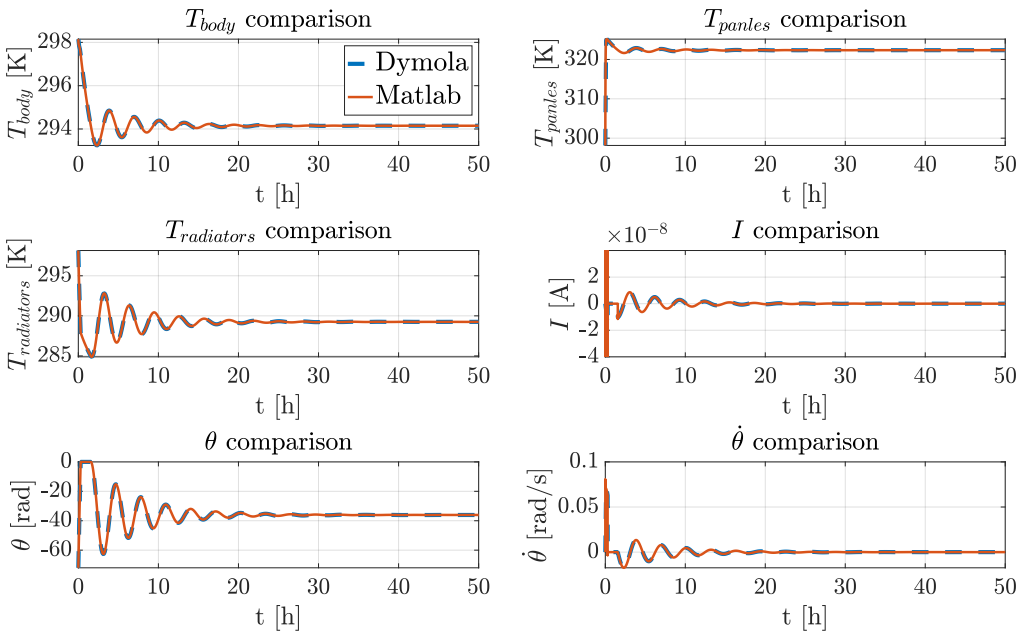

Figure 13: Comparison between casual and acasual model

The results demonstrate that the two methods produce almost identical solutions, as evidenced by the overlap of the plots. The evolution of the temperature for all the nodes is the same compared to the one computed with the causal modeling approach, as the evolution of θ, $\dot{\theta}$ and the current I. Consequently, the robustness of the implemented control logic block is validated. One last consideration can be done on the implementing difference between the two approaches. The implementation in Dymola proved to be more straightforward, particularly for the active control of the radiator. The control logic was easily developed using logical switch blocks, highlighting the simplicity and efficiency of the approach in this environment compared to the Matlab scripts.

Exercise 2

A simplified conceptual map of the Attitude Control System (ACS) of a satellite is depicted in Figure 14. For the accuracy of some measurements that the spacecraft has to obtain, the probe has to fly in an orbit very close to the Earth's surface. The spacecraft is therefore subjected to a high atmospheric drag. The ACS has to compensate for the deceleration caused by this drag continuously. In general, ACSs are made of 3 sensors in orthogonal directions and 3 thrusters to detect and compensate for linear and angular accelerations on the probe in each direction, but for the sake of this project, only compensation in the tangential direction will be considered.

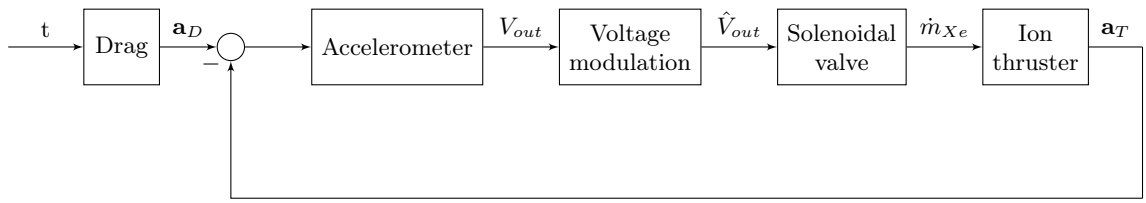

Figure 14: Block diagram of the ACS.

The ACS is modeled with several modules: an **accelerometer** measuring the accelerations acting on the spacecraft, a **voltage modulation block** controlling the output voltage, the **control valve** modifying the aperture of the thruster valve, and the **ion thruster**.

The accelerometer, kept aligned with the direction of velocity, presents inside a seismic mass that is subject to an external acceleration $\mathbf{a_D}$ and $\mathbf{a_T}$ given only by the drag D and the thrust T, respectively. The mass is in between the stators of a condenser and, moving for the acceleration given by these two forces changes the output voltage V_{out} of the circuit of the accelerometer. The dynamics of the seismic mass itself can be modeled as a mass-spring-damper system as follows

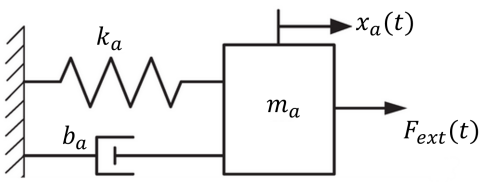

Figure 15: Model of the accelerometer.

$$\dot{x}_a = v_a \tag{8}$$

$$F_{ext}(t) = \frac{T - D}{M_{SC}} m_a = m_a \dot{v}_a + b_a v_a + k_a x_a \tag{9}$$

where M_{SC} is the mass of the spacecraft, m_a is the seismic mass value, and b_a and k_a are respectively the damper and the spring terms present in the accelerometer. For the causal modeling, consider the V_{out} as directly proportional to the velocity of the seismic mass, as $V_{out} = K_{acc} v_a$ [1].

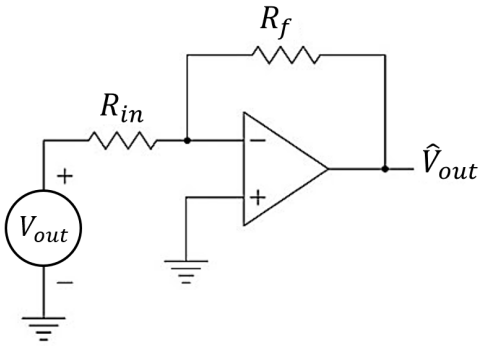

Figure 16: Inverting operational amplifier.

The output of the accelerometer V_{out} needs to be modulated into $\hat{V}_{out}$ to adjust the control for the solenoidal valve. This modulation is performed through an operational amplifier, displayed in Figure 16. The operational amplifier in inverting configuration modifies V_{out} into $\hat{V}_{out}$ accordingly to:

$$\hat{V}_{out} = -\frac{R_f}{R_{in}} V_{out} \tag{10}$$

where R_f and R_{in} are the two resistances modifying the input voltage.

The voltage $\hat{V}_{out}$ is itself the input of the solenoidal valve, shown in Figure 17, and creates a current I commanding the flow control valve. The current passes through a circuit modeled with only a solenoid with a variable inductance $L(x_v)$. As the current variably flows, a magnetic field is generated. A resulting force f_v acts on the spool, which as a result moves.
The armature-spool arrangement is then modeled again as a lumped parameter spring-mass-damper system

$$\dot{x}_v = v_v \tag{11}$$

$$m_v \dot{v}_v = -k_v x_v - b_v v_v + \frac{1}{2} I^2 \frac{dL}{dx_v} \tag{12}$$

$$\dot{I} = \frac{1}{L(x_v)} \hat{V}_{out} \tag{13}$$

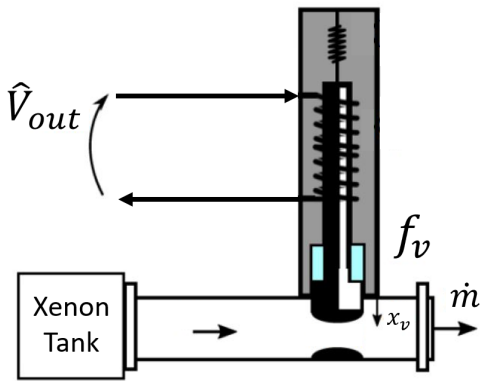

Figure 17: Model of the solenoidal valve.

where x_v and v_v represent the position and velocity of the spool, and b_v and k_v are respectively the damper and the spring terms, and m_v is the mass of the spool. The dynamics of the current I is generated by $\hat{V}_{out}$ through the variable inductance $L(x_v) = \frac{1}{\alpha+\beta x_v}$, depending on the position of the valve itself.
As the spool moves, the area $A_v(x_v)$ of the duct through which a Xenon flow is passing changes. The resulting mass flow rate $\dot{m}_{Xe}$ of Xenon is ejected into the ion thruster, shown in Figure 18. In particular, the area of the valve can be modeled as

$$A_v(x_v) = A_0 + \ell(x_{v,\max} - x_v) \tag{14}$$

In particular, the orifice area $A_v(x_v)$ has been modeled as linearly varying with x_v. The tool controlling the area is a flap of width ℓ and maximum extension $x_{v,\max} = \ell$, for simplicity.
The value A_0 is the minimum area of the orifice, such that:

$$0 \leq x_v \leq x_{v,\max}$$

$$\underbrace{A_0 = A_v(x_{v,\max})}_{\text{minimum area}} \leq A_v \leq \underbrace{A_v(0) = A_0 + \ell^2}_{\text{maximum area}} \tag{15}$$

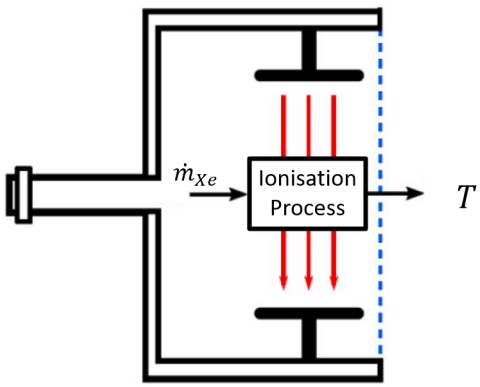

Figure 18: Model of the ion thruster.

The ion thruster comprises an ionization chamber that utilizes a strong magnetic field, ionizing the Xenon flux. The generated ions are moved towards the acceleration grid, which accelerates the ions using a strong electric field, producing a thrust on the spacecraft as the final effect. The flux of Xenon $\dot{m}_{Xe}$ can be expressed as follows

$$\dot{m}_{Xe} = A_v(x_v)\sqrt{k\rho_{Xe}p_T(\frac{2}{k+1})^{\frac{k+1}{k-1}}} \tag{16}$$

where $\rho_{Xe} = \dfrac{p_T}{\bar{R}T_T}$, k and $\bar{R}$ being respectively Xenon specific heat ratio and gas constant. In the tank, the gas is at a total pressure of p_T and temperature of T_T. The Xenon flow enters the

thruster and is here ionized into ions with mass m_i and charge q. After the acceleration imposed by the acceleration grid with a voltage ΔV, the ions exit from the nozzle with a velocity v_{exit}, producing the thrust T, as follows

$$T = \dot{m}_{Xe} v_{ext} = \dot{m}_{Xe} \sqrt{\frac{2q\Delta V}{m_i}} \tag{17}$$

Table 4 reports the values for the parameters to simulate the ACS system, with symbol and unit of measure. The drag $D(t)$ can be modeled as a function of time only, as it follows

$$D(t) = 2.2 - \cos(\omega_s t) + 1.2 \sin(\omega_o t) \cos(\omega_o t) \tag{18}$$

where t enters in seconds, and $D(t)$ is in mN.

Component	**Parameter**	**Symbol**	**Value**	**Unit**
Accelerometer	Spacecraft mass	M_{SC}	300	kg
	Seismic mass	m_a	0.32	kg
	Accelerometer damper	b_a	$[1.5{\cdot}10^3$–$2{\cdot}10^4]$	Ns/m
	Accelerometer spring	k_a	$[5{\cdot}10^{-5}$–$3{\cdot}10^{-3}]$	N/m
	Acc. proportional coefficient	K_{acc}	1	Vs/m
Amplifier	Inverting resistance	R_{in}	[0.1–10]	Ω
	Feedback resistance	R_f	$[1{\cdot}10^4$–$8{\cdot}10^4]$	Ω
Solenoidal Valve	Spool mass	m_v	0.1	kg
	Valve spring	k_v	$1{\cdot}10^3$	N/m
	Valve damper	b_v	$1{\cdot}10^3$	Ns/m
	Solenoid constant	α	$2.1{\cdot}10^{-2}$	1/H
	Solenoid gain	β	-60	1/Hm
	Minimum area	A_0	$4.7{\cdot}10^{-12}$	m^2
	Maximum extension	$x_{v,\max}$	$1{\cdot}10^{-5}$	m
Thruster	Heat ratio	k	1.66	-
	Tank pressure	p_T	$2{\cdot}10^5$	Pa
	Tank temperature	T_T	240	K
	Gas constant	$\bar{R}$	63.32754	J/kg K
	Charge	q	$1.6{\cdot}10^{-19}$	C
	Voltage	ΔV	2000	V
	Ion mass	m_i	$2.188{\cdot}10^{-25}$	kg
Drag	Secular pulsation	ω_s	$1.658226{\cdot}10^{-6}$	rad/s
	Orbital pulsation	ω_o	$1.160758{\cdot}10^{-3}$	rad/s

Table 4: Parameters for the simulation of the ACS.

Part 1: causal modeling (9 points)

Reproduce in Matlab the physical model of the ACS, in particular

1. Formulate the full system of non-linear ODEs making explicit the state variables of the whole simulation. Tune the values of the parameters not explicitly stated in Table 4 to have the compensating thrust T the most similar to the disturbing drag D.

2. Choose the appropriate initial conditions and simulate for three orbital periods T_o, where $T_o = 2\pi/\omega_o$.

3. Discuss the selection of the ODE integration scheme.

Part 2: acausal modeling (6 points)

Reproduce in Simscape the physical model of the ACS. Choose the appropriate initial conditions and parameters, simulate for three orbital periods T_o, and compare the results with those obtained in the prior point.
Note: when modeling the solenoid valve in Simscape, please consider the nominal values shown in Table 5.

Parameter	Value	Unit
Pull-in forces [F_1 - F_2]	[9000 - 12]	N
Stroke [x_1 - x_2]	[0 - 0.1]	mm
Maximum Stroke	0.1	mm
Rated voltage	0.6	mV
Rated current	0.1	A
Contact stiffness	0.12×10^6	N/m
Contact damping	10^4	Ns/m

Table 5: Solenoid parameters in Simscape.

(15 points)

Point 1.1

The Attitude Control System (ACS) to be developed has to compensate for the deceleration caused by the atmospheric drag continuously. To formulate the full system of non-linear ODEs, each block of the ACS has been analyzed, and the correlated state variables have been properly defined.

1. **Accelerometer**: the accelerometer dynamics is provided in Eq. (8) and Eq. (9), where x_a and v_a are the state variables.

2. **Voltage modulation**: the accelerometer output V_{out} is considered directly proportional to v_a, and then modified into $\hat{V}_{out}$ as in Eq. (10).

3. **Solenoid valve**: the armature-spool arrangement model is stated in Eq. (11) and Eq. (12), where x_v and v_v represent the state variables. The spool dynamics is influenced by the current I, last state variable, whose variation depends on the solenoid inductance L_{x_v} and the modulated $\hat{V}_{out}$ from the amplifier, as provided in Eq. (13).

4. **Ion thruster**: the flux of Xenon $\dot{m}_{Xe}$ can be expressed as in Eq. (16). In the worst-case scenario where the valve is in full open position, the computed mass flow rate is $0.123\frac{mg}{s}$. In the given simulation time of three orbital periods, the total amount of expelled Xenon mass would be equal to $2g$, which is negligible with respect to the $300kg$ satellite mass. Thus, the satellite mass has been assumed constant and $\dot{m}_{Xe}$ was not included in the state variables.

Given these considerations, the system consists of five state variables, which are as follows:

$$\mathbf{x} = [x_a; v_a; x_v; v_v; I]$$

Now, to retrieve the full system of ODEs, it is necessary to express the modulated voltage $\hat{V}_{out}$ as function of v_a and to compute the derivative of the inductance L_{x_v} with respect to x_v. These intermediate passages are formulated in Eq. (19).

$$\hat{V}_{out} = -\frac{R_f}{R_{in}} V_{out} = -\frac{R_f}{R_{in}} k_{acc} v_a \qquad and \qquad \frac{dL(x_v)}{dx_v} = -\frac{\beta}{(\alpha + \beta x_v)^2} \tag{19}$$

The expressions of the thrust T and the drag D are given in Eq. (17) and in Eq. (18). For the sake of clarity, they have not been substituted into the system.
The full system of non-linear ODEs can finally be recovered, as reported in Eq. (20)

$$\begin{cases} \frac{dx_a}{dt} &= v_a \\ \frac{dv_a}{dt} &= \frac{T-D}{M_{sc}} - \frac{b_a}{m_a} v_a - \frac{k_a}{m_a} x_a \\ \frac{dx_v}{dt} &= v_v \\ \frac{dv_v}{dt} &= \frac{1}{2} \frac{-\beta}{m_v(\alpha+\beta x_v)^2} I^2 - \frac{b_v}{m_v} v_v - \frac{k_v}{m_v} x_v \\ \frac{dI}{dt} &= -\left(\alpha + \beta x_v\right) \frac{R_f}{R_{in}} k_{acc} v_a \end{cases} \tag{20}$$

As for the tuning of parameters, to have the compensating thrust as close as possible to the drag at each time step, the system dynamics speed has to be maximized.
Starting with the accelerometer, the selected values are $k_a = 3 \cdot 10^{-3}$ and $b_a = 1.5 \cdot 10^3$ to guarantee the fastest possible response of the system. In fact, this configuration minimizes damping b_a to avoid filtering out real dynamics while maximizing stiffness k_a to limit noise.
Regarding the operational amplifier, it is necessary to ensure a more responsive drag compensation through its amplification factor. In fact, the thrust is directly influenced by the mass flow rate, which affects the position of the spool x_v. The rate of change v_v of the spool position is maximized if the input voltage $\hat{V}_{out}$ to the solenoid is maximized. Consequently, the amplifier resistances must be selected to guarantee the maximum gain: $R_f = 8 \cdot 10^4 \Omega$ and $R_{in} = 0.1\Omega$.

Point 1.2

The first approach to the solution is carried out with general initial conditions without any refinement. This allows testing the ability of the ACS to meet performance requirements starting from a switched-off control, which may be far from equilibrium.

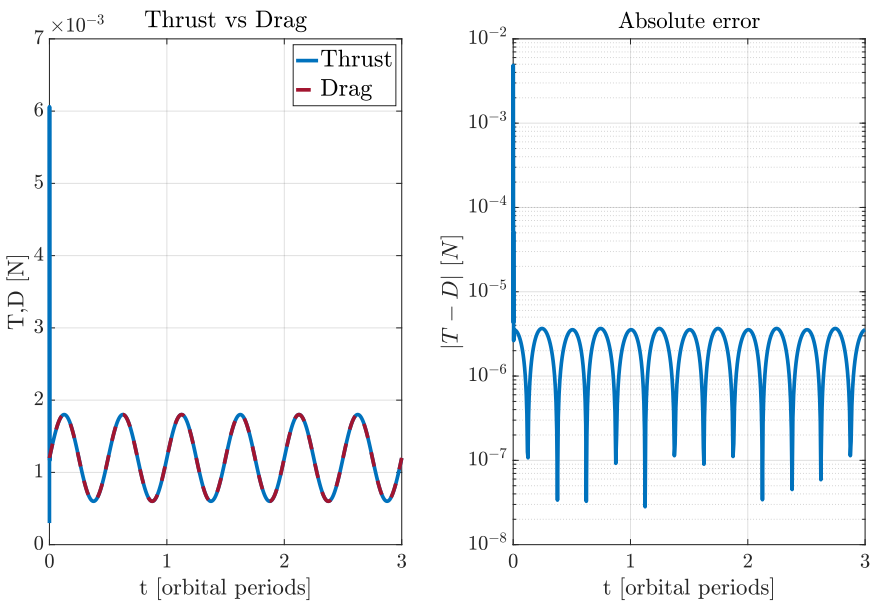

Figure 19: Thrust vs Drag evolution with general initial conditions

At the beginning of the simulation, the seismic mass is assumed to be at rest, resulting in a zero output voltage V_{out}, as it is directly proportional to the mass velocity v_a. Consequently, the current I is also initially zero, as it is proportional to the modulated voltage $\hat{V}_{out}$. Finally, for the spool arrangement, it is assumed that the orifice is fully open at the beginning, so $x_v = l$ and $v_v = 0$. Thus, the initial guess was set to $\mathbf{x} = [0; 0; l; 0; 0]$.
The resulting behavior of the system is shown in Figure 19. As observed, the initial spike in the thrust indicates that the starting condition is far from the equilibrium solution, suggesting that further optimization of the initial guess is necessary. In the end, after a short initial transient, the error settles to a very low value of approximately $10^{-6}N$, three orders of magnitude lower than the average drag value. Thus, the thrust efficiently compensates for the atmospheric drag, demonstrating that the ACS works as intended.
The spikes are also present in the evolution of each state variable, as presented in Figure 20. After these initial peaks, all variables exhibit oscillatory behavior. Focusing on the position of the spool x_v, it is evident that it remains within the physical boundaries, starting from the maximum length imposed by the initial condition. It is also noticeable that its minimum points correspond to the maximum points of the thrust profile. This behavior reflects the relationship between thrust and spool movement, where an increase in thrust leads to the spool opening further.

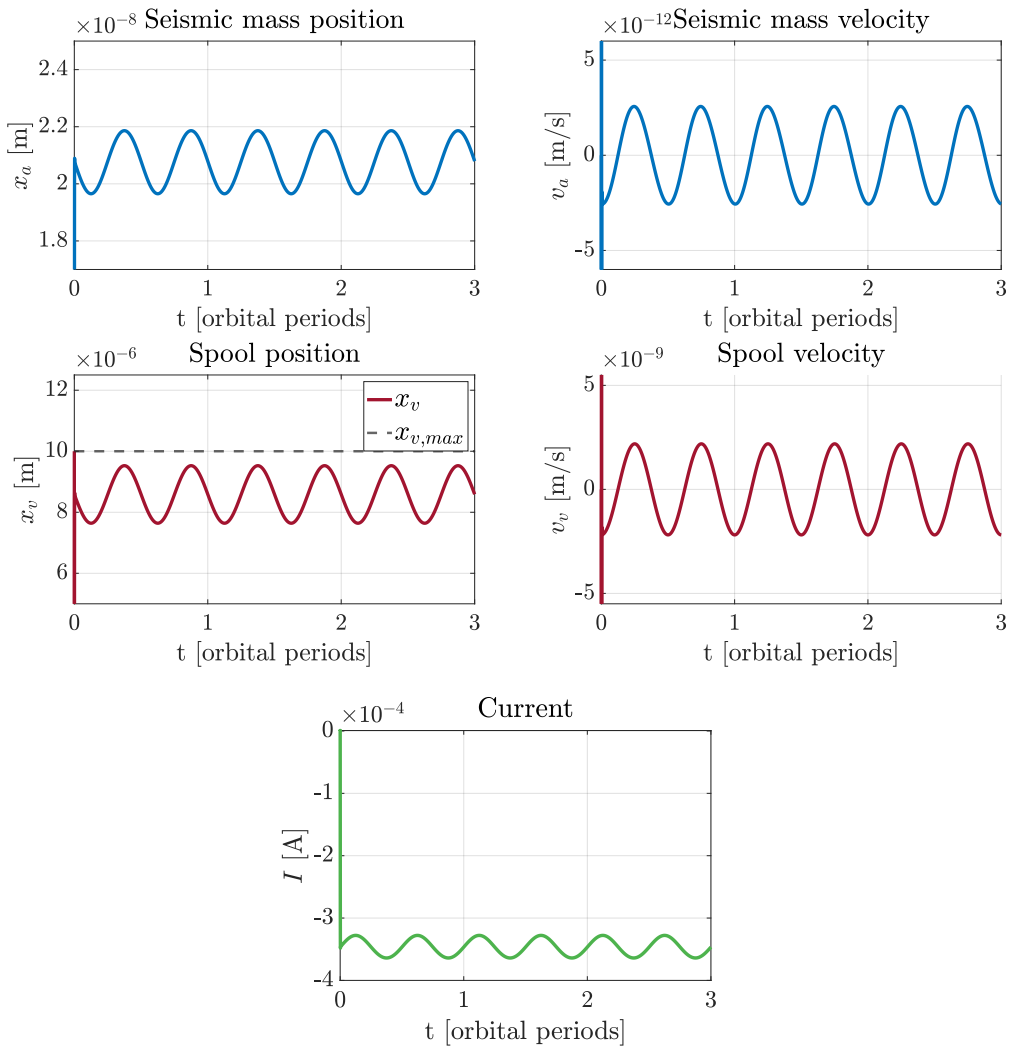

Figure 20: State variables evolution with general initial conditions

To appreciate the transient phase of the system, only the transient response dynamics is reported in Figure 21, since it is very fast with respect to the simulation time. Although a peak is observed at the beginning of the simulation due to the initial condition being far from equilibrium, it is clear that the absolute error quickly drops to the previous asymptotic value of $10^{-6}N$. The transient response of the system is very rapid and convergence is reached after approximately 15 seconds. This rapid decrease highlights the rapid response of the ACS system and its ability to efficiently correct the mismatch.

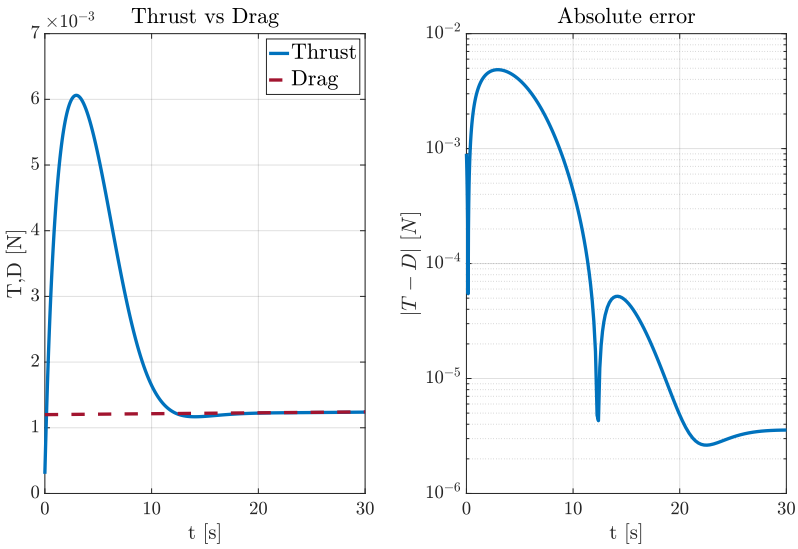

Figure 21: Thrust vs Drag evolution during transient phase

Finally, also the transient phase of all state variables is shown in Figure 22 and Figure 23. As it can be seen in the x_a plot, the valve is opened, from its close position, to balance the discrepancy between drag and thrust. Lastly, it is important to note that the periodic oscillations are not visible when convergence is reached, since the simulation time is very small with respect to the oscillation period.

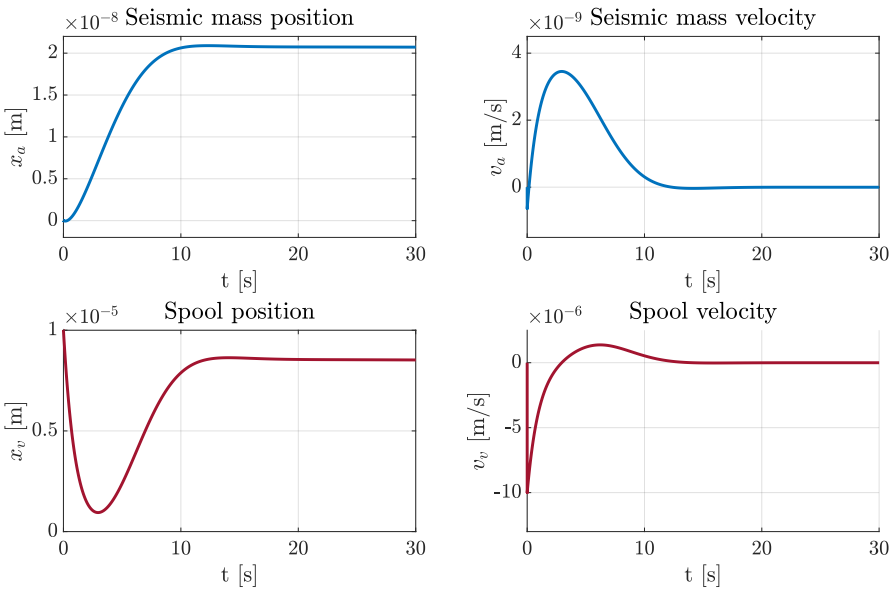

Figure 22: State variables evolution during transient phase

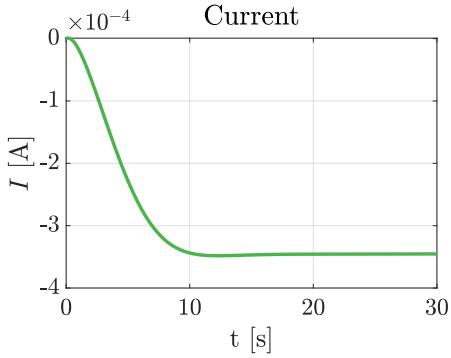

Figure 23: Current evolution during transient phase

Point 1.2.bis

In this second method, a refinement of the initial condition is performed to avoid initial spikes, ensuring that the simulation starts from a configuration that perfectly compensates for the drag. This approach aims to optimize the initial guess to achieve a smoother and more stable simulation, and evaluate the performances of the ACS at steady state.
For the optimization of the initial condition approach, a while loop was implemented. Within this loop, the ODEs are solved, and the absolute error between thrust and drag is computed. If the error between two successive iterations is below a threshold of 10^{-6}, the loop terminates, indicating that the system has converged. Otherwise, the loop repeats with an updated initial condition based on the final state from the ODEs. After just a few iterations, the initial guess that minimizes the absolute error was found to be:

$$\mathbf{x} = [2.0794 \cdot 10^{-8}; -2.5591 \cdot 10^{-12}; 8.5891 \cdot 10^{-6}; -2.1883 \cdot 10^{-9}; -3.4657 \cdot 10^{-4}]$$

Using this optimized initial guess, Figure 24 demonstrates how the initial spike is no longer present and the absolute error remains consistently lower than before during the first iterations. Thanks to this method, it is possible to analyze the behavior of the ACS under steady-state conditions more effectively.

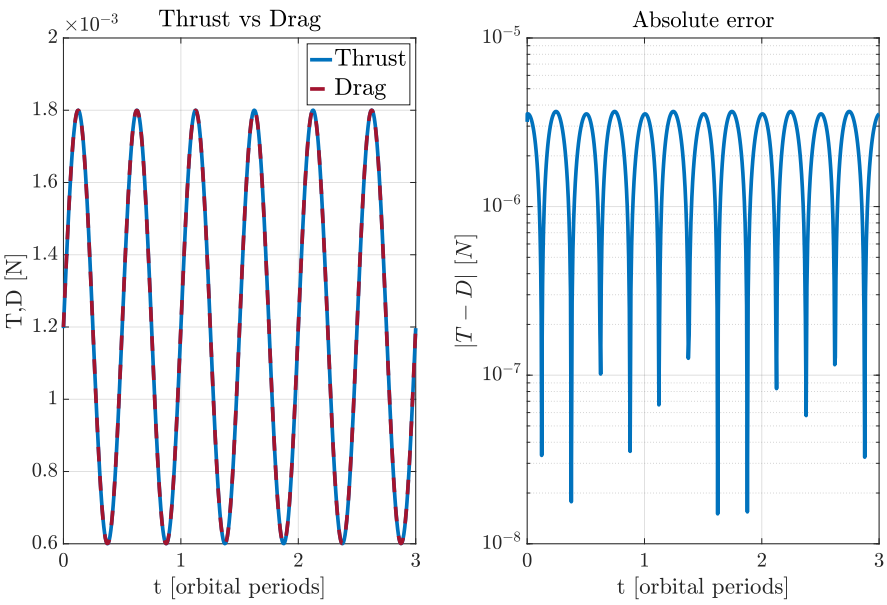

Figure 24: Thrust vs Drag evolution with optimized initial conditions

Finally, as clearly shown in Figure 25, all state variables, except the accelerometer mass, behave exactly as they would if the system had started directly under general conditions (see Figure 20), once the transient phase has passed. This is because the spool position and the solenoid behavior do not directly depend on the seismic mass position, but rather on the voltage generated by the differential capacitive accelerometer, which solely depends on the seismic mass velocity. The value around which x_a oscillates changes based on the initial discrepancy between T and D.

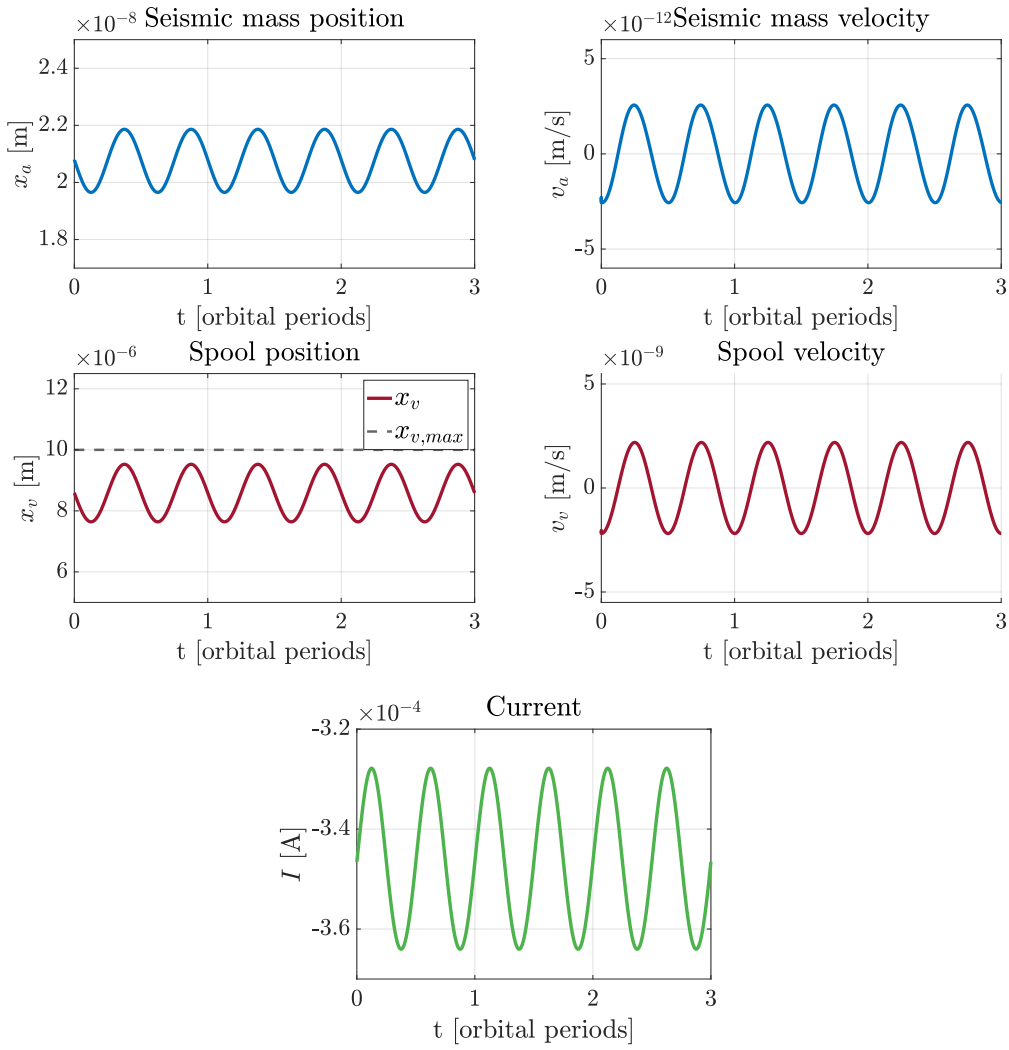

Figure 25: State variables evolution with optimized initial conditions

Point 1.3

The system is expected to be stiff since multiple physical domains with different dynamics speeds are present. For this reason, the selected Matlab© solver is `ode15s`, which is a variable step integrator capable of dealing with stiff dynamics. The solver was configured with a relative and absolute tolerance of 10^{-12} to ensure accurate results.

Point 2

The acausal modeling of the ACS physical system has been carried out using Simscape libraries in Simulink environment. To properly set up the acausal simulation, the implementation of the various blocks has been divided into the relevant physical domains and configured with the parameters provided in the exercise.

1. **Accelerometer** The accelerometer was modeled using a *Mass* block connected to a *TranslationalSpring* and a *TranslationalDamper* block, both in parallel, obtaining a mass spring damper system. A *MechanicalTranslationalReference* block was connected to these elements to represent a reference point. An *IdealForceSource* source is connected to the seismic mass, together with a position and velocity sensor (*IdealTranslationalMotionSensor*) to recover the seismic mass state (x_a and v_a) during the simulation.

2. **Voltage modulation** The *TranslationalElectromechanicalConverter* block was used to provide an interface between the electrical and mechanical domain, connecting the accelerometer to the operational amplifier system. The voltage modulation was modeled with a *Op-Amp* block and two *Resistor* blocks. All electrical components were connected to the ground, represented by the *ElectricalReference* block. The circuit has been built taking as reference Figure 16. Finally, a *CurrentSensor* was then added in series to measure the current I.

3. **Solenoid valve** The solenoid valve dynamics was introduced in the model using the *Solenoid* block, which embedded the equation of motion of a plunger under an exciting electromagnetic force. An *IdealTranslationalMotionSensor* was employed to measure the spool position x_v and velocity v_v.

4. **Ion thruster** The Ion thruster physical system was not modeled using an acausal approach since it is very complex. Thus, it was modeled using Matlab function within Simscape: the equations describe the mass flow rate as function of the spool position and the thrust computation.

Finally, drag was also modeled using Matlab function, and the difference between these forces, multiplied by the ratio $\frac{m_a}{M_{SC}}$, was provided as an input to the *IdealForceSource* block, which modeled the external force acting on the seismic mass.
The complete Simscape model is reported in Figure 26, where the interaction between the acausal and causal blocks is clearly visible.

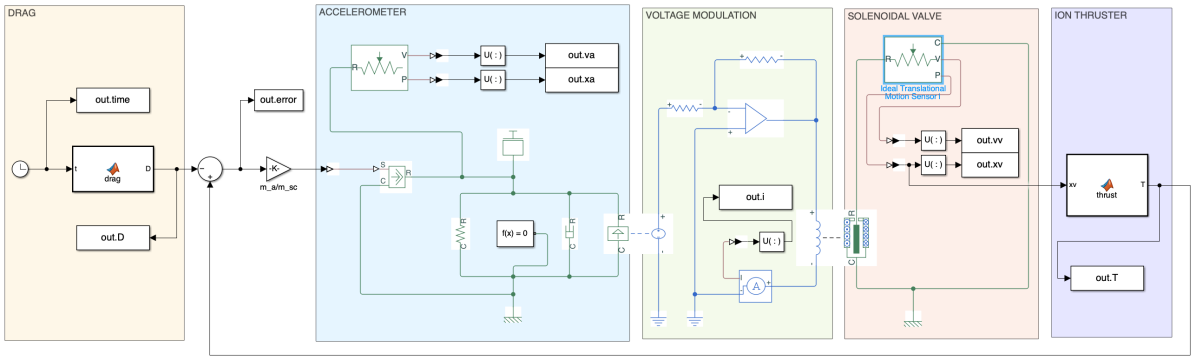

Figure 26: Simscape ACS model

The integration has been performed using `ode15s`, since it is capable of integrating stiff systems. Moreover, it is the same scheme that is used in Matlab for the causal modeling solution. The relative tolerance of the solver has been set equal to 10^{-10}.

The system is very sensitive to initial conditions and diverges for any initial state different from $\mathbf{x} = [0; 0; 0; 0; 0]$, representative of a system at rest and spool in the fully open position at the beginning of the simulation. Since the acausal simulation starts from a null state vector, to have a coherent comparison with the causal approach, also the Matlab solution has been computed again starting from these new initial conditions.
The resulting behavior of the system is presented in Figure 27. As observed, the initial spike is back along with the non-optimal initial condition, but convergence is still reached, demonstrating that the ACS acasual model efficiently compensates for the atmospheric drag.
However, the error is significantly larger compared to the results obtained with causal modeling: the system is slower in keeping up with the drag variation. The error obtained in Simscape is in the order of $10^{-2}mN$, while the one in Matlab is in the order $10^{-3}mN$. Furthermore, the difference between thrust and drag oscillates around a zero mean value in the solution of the causal approach. On the contrary, the computed thrust in Simscape is always greater than the drag, thus the error has a non-zero mean value. Then a small acceleration acts on the system in the same direction for the entire simulation.

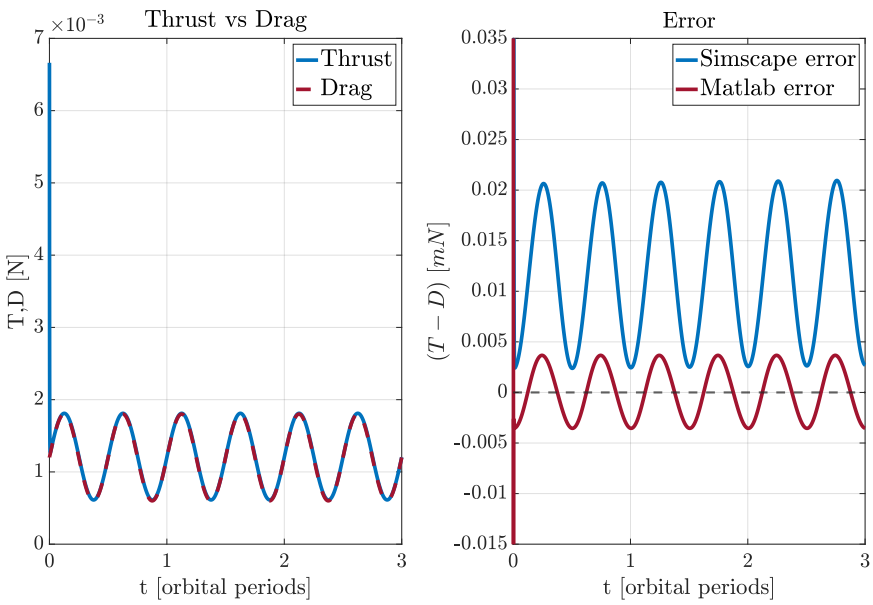

Figure 27: Thrust vs Drag evolution in Simscape and comparison with Matlab

This acceleration direction is reflected in the accelerometer dynamics, which behaves differently from the solution obtained in Matlab, as shown in Figure 28. The position x_a of the seismic mass exhibits a divergent trend, rather than an oscillatory behavior. Consequently, the velocity v_a has oscillations of higher amplitude with respect to the Matlab solution, consistent with the higher error between thrust and drag obtained in the acausal modeling approach. Therefore, also the evolution of the current I is affected: the current magnitude is higher.
On the other hand, the observed trend of x_v and v_v match the one obtained with Matlab. In fact, also the compensating thrust is similar, since it is function of x_v and is computed using causal blocks that implement the same equations exploited in the causal part.
One of the main reasons of the discussed discrepancies could arise from the different behavior of the *Solenoid* block in Simscape, with respect to the characteristic equations implemented in Matlab where the solenoid parameters were not explicitly modeled.
In conclusion, the acausal modeling solution can be considered validated. The discrepancies with respect to the casual model can be attributed to the high sensitivity of the solenoid valve to the input parameters.

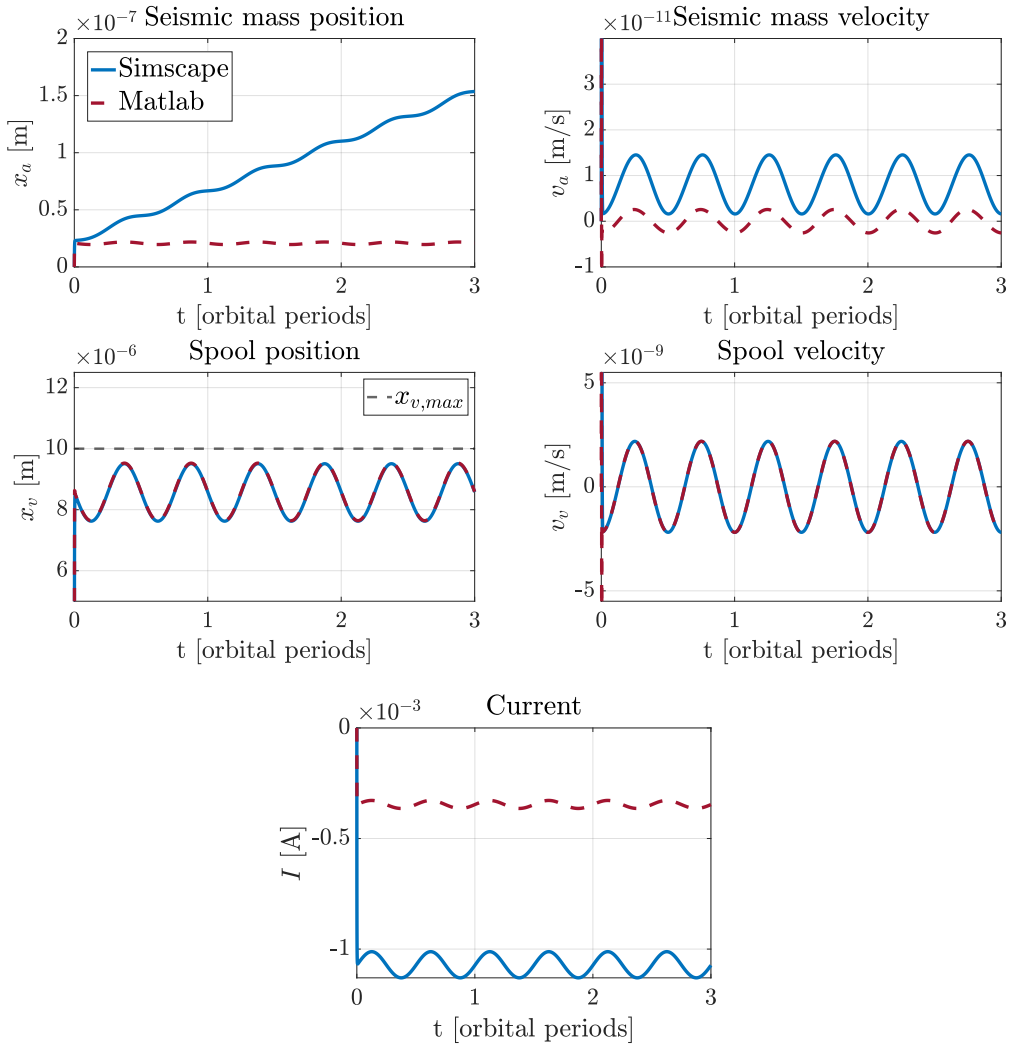

Figure 28: State variables evolution comparison between Simscape and Matlab

References

[1] R. Mukhiya, M. Garg, P. Gaikwad, et al. *Electrical equivalent modeling of MEMS differential capacitive accelerometer*. In: Microelectronics Journal. Vol. 99, pp. 104770, 2020. `https://doi.org/10.1016/j.mejo.2020.104770`